

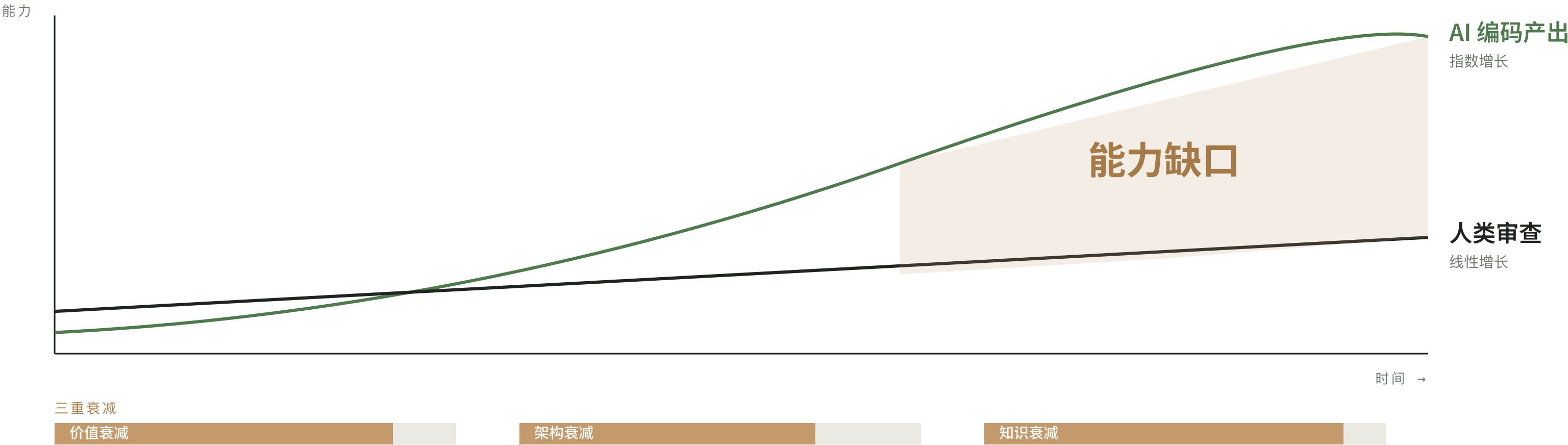
A MANIFESTO · 2026

让 AI 编程 从加速器 变为解放者

三层引力梯度 · 三次能力跃迁 — 走向 AI 自主编程的工程之路

当下困局

代码生成已超越人类的验证能力



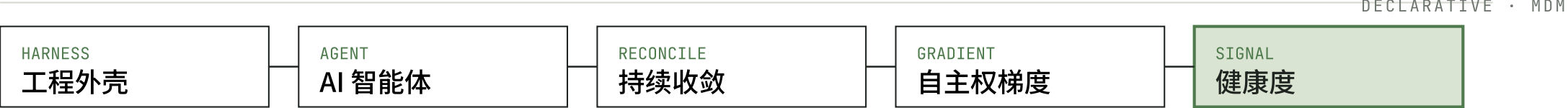
不能让 AI 单纯加速 — 必须把 AI 装进可信的工程系统

总图 · 全程使用同一案例：订单服务支持团购功能

引力梯度 · 三层 × 三跃迁



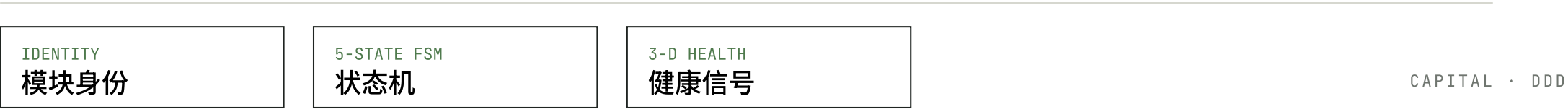
EXPANSION · 跃迁③ 内部



EXPANSION · 跃迁② 内部



EXPANSION · 跃迁① 内部



这张图 = 后续每一页的"全图地图" — 每页都会高亮其中一处

前置条件 · 不是阶段

工程引力场

把 AI 装进系统之前，必须先把"系统"建出来 · 这是物理学约束

YOU ARE HERE

L0

01 · 框架

机床 · 领域抽象

领域边界 · 契约骨架 · 质量基底

↳ 框架越好 · AI 价值越大

02 · 模块身份

资产可识别

module.yaml · 身份 · 意图 · 状态 · 信号

↳ L1 跃迁的入口

03 · CI/CD 门禁

质量内建

lint · type · test · contract · sec scan

↳ 左加速 100× · 右重武装 3×

04 · 运行时 · NEW

可运行环境

容器 · K8s · service mesh · registry

↳ AI 写的代码必须能跑起来

05 · DevOps 全栈 · NEW

部署 · 观测 · 回滚

deploy · log · trace · metric · secret · alert

↳ AI 错了 · 必须能秒级回滚

06 · 沙箱

受控试错

隔离 · 可观测 · DDD 边界上下文

↳ AI 只在沙箱中尝试



基底建好 · 跃迁才能逐层叠加 — 跳过 L0 = 把 AI 放进沼泽

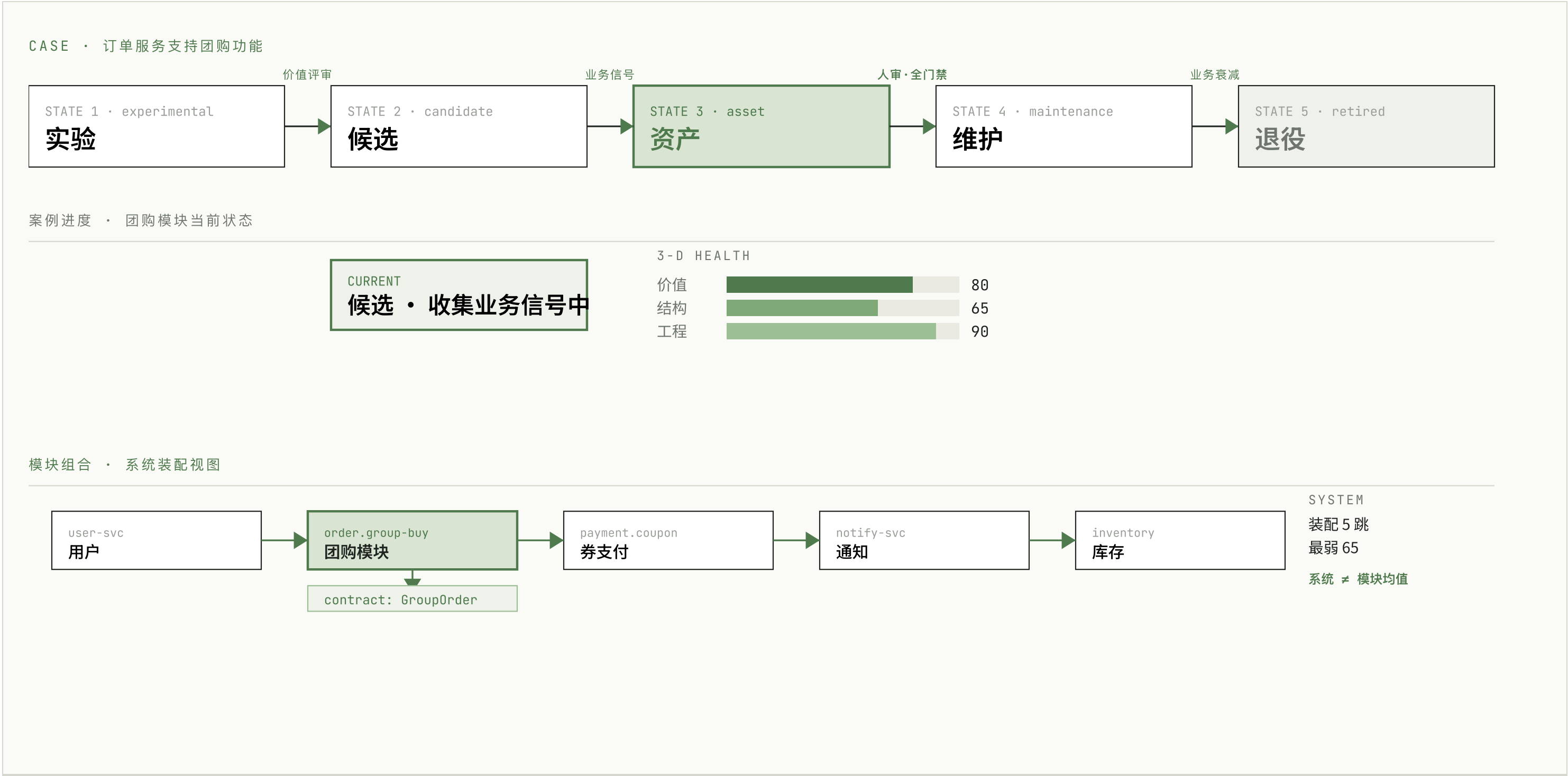
→ L1 状态可见

从代码 → 状态

两个维度 · 单模块生命周期 + 模块间装配关系 — 只有模块 · 没有系统装配 · 等于系统工程失败

YOU ARE HERE

L1



输入 来自 L0 框架与门禁 · 输出 给 L2：可被声明的状态

→ L2 意图可表达

递进关系 · 桥

看见状态 → 才能描述意图

L1 把每个模块变成"可观测对象" · L2 把每个变更变成"可声明事件"

L1 OUTPUT · 信号

order-service.discount · 调用率 ↓ 73% / 30d

order-service.group-buy · 错误率 2.1%

payment.coupon · 健康度 91 / 90 / 88

...数百模块的实时画像

单纯信号 → 是数据洪流，没有方向



L2 INPUT · 意图

INTENT · ORDER-SERVICE / GROUP-BUY

"让订单服务支持团购，
支付走券码，p99 < 200ms"

↓ AI 翻译 ↓

contract: POST /orders/group
sla.p99: 200ms · sla.error: 0.5%
depends_on: payment.coupon (≥85)
success: signals.usage ≥ 1k/d

没有 L1 的状态可见 · L2 的意图就只是空头支票

从命令 → 意图

不是把 MDM 套用过来 · 是把"声明 + 收敛"的思想范式重新落到代码层面

INTENT FILE · ORDER-SERVICE / GROUP-BUY

01 · BUSINESS

业务意图

goal: 团购下单
metric: GMV ↑ 15%
deadline: 2026-Q2

02 · CONTRACT

技术契约

api: POST /orders/group
schema: GroupOrder
events: [GroupCreated]

03 · QUALITY

质量阈值

p99: 200ms
error: < 0.5%
coverage: ≥ 95%

04 · LIFECYCLE

生命周期

state: candidate
sunset_if: usage < 100/d
review_at: 30d

这就是 Manifest 的真容 — 不是抽象概念，是这四块

不是文档 · 是可执行

每一字段都直接映射到验证器

p99 → 性能测试 · contract → 合约测试 · sunset_if → 自动告警

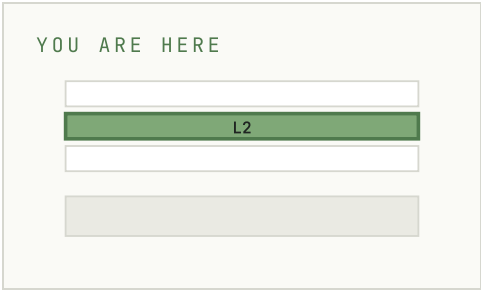
MDM 创造性应用

设备 → 软件模块

设备拉取 declaration · 模块拉取 intent；设备配置自适应 · 模块自演化。同构而不是同物。

解决什么业务需求

团购需求不再是 PRD 文档 + 排期会 + Jira 票，而是一份 intent 文件 — 一边声明、一边交付、一边监控。



递进关系 · 桥

声明 → 才需要"执行者"

L1 状态机给出"现在哪里" · L2 意图给出"要去哪" · L3 Harness 是"自动化的去那里"



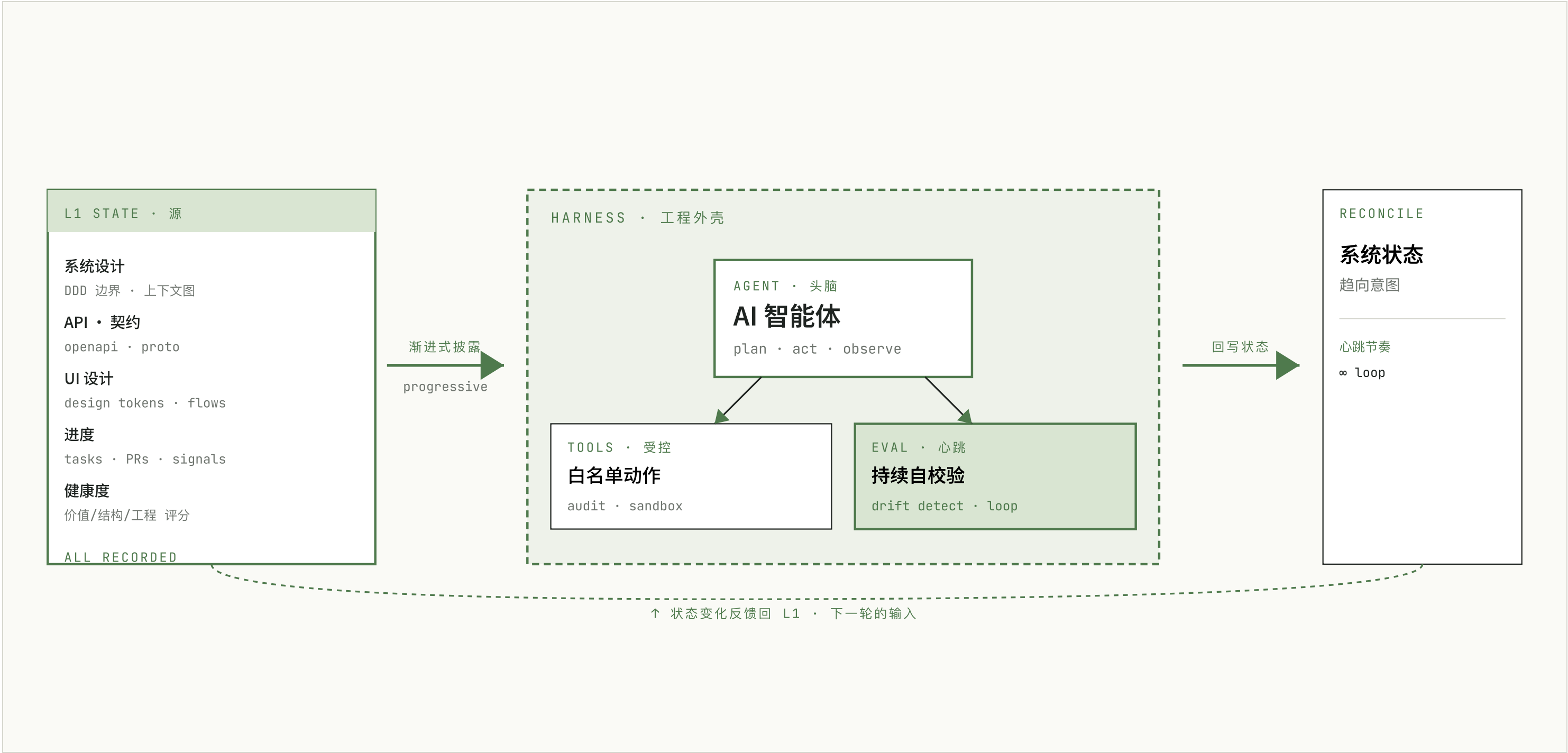
关键：循环不是"一次推理" · 是"持续到达成意图为止"

从一次推理 → 持续自治

Agent 的上下文与进度 · 大部分可从 L1 系统状态渐进式披露 · 不必凭空构造

YOU ARE HERE

L3



关键 不是更聪明的 AI · 是更稳定的"外壳 + 心跳"

→ AUTONOMY GRADIENT

不是"AI 全权" · 是"AI 接管可逆动作"

REVERSIBILITY · 可逆度

可逆 ←

→ 不可逆

R0 · READ	R1 · LOCAL	R2 · CONTROLLED	R3 · CROSS	R4 · USER IMPACT	R5 · IRREVERSIBLE
只读分析 查代码 · 提建议 AI 全自动 no human	本地修改 改本仓库 · 单测护栏 AI 自动 git rollback	受控外部 沙箱 API · 测试环境 AI 自动放行 audit log	跨域写入 改其他服务 · 迁移 AI 提议 + 人审 staged rollout	影响用户 删数据 · 改账单 人审 · 永不放开 human gate	资金 · 物理 转账 · 设备控制 人决策 · 红线 never auto

把可逆动作交给 AI · 把不可逆动作留给人 — 这才是真正的解放

总分关系总览

全景闭环 · 团购功能从声明到上线

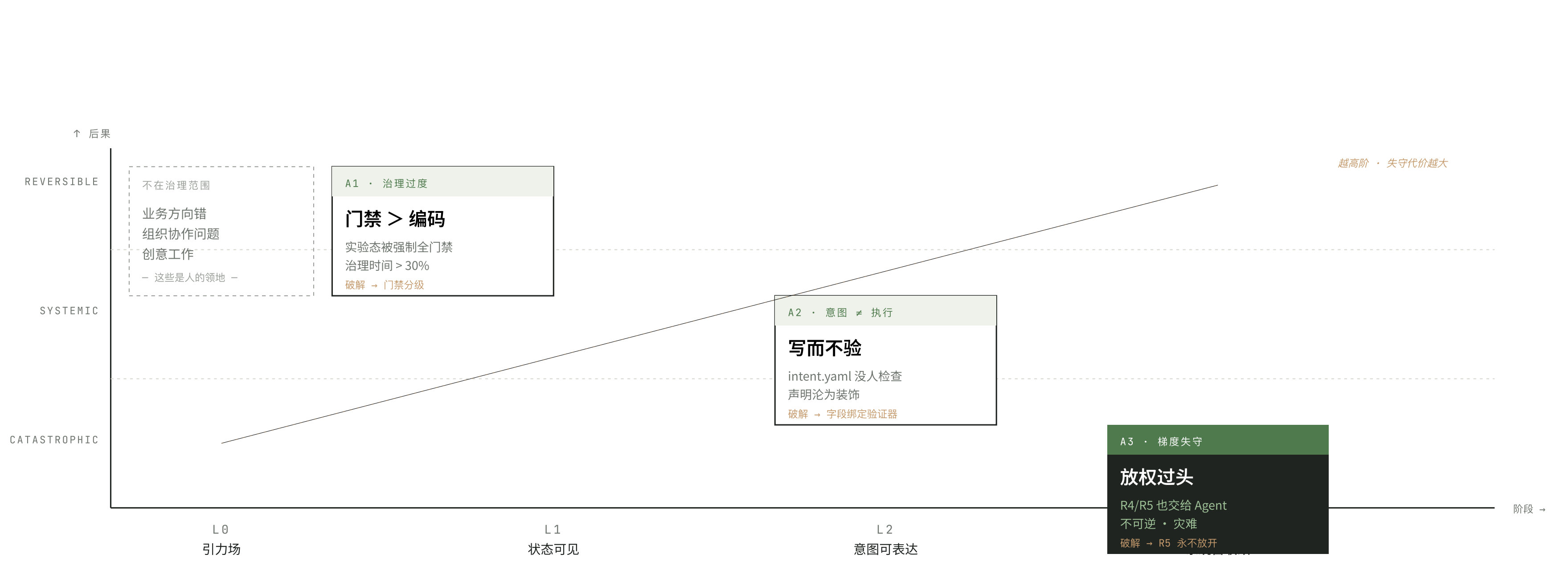


L0 → L1 → L2 → L3 不是阶段，是协作 — 同一时间，每一层都在工作

边界

反模式 · 何时该停手

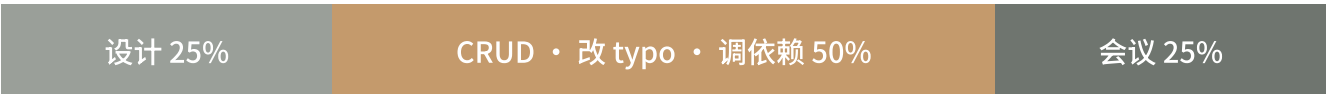
每个跃迁都有失守模式 · 知道边界比知道方法更重要



人的位置

劳动力流向更高价值的地方

TODAY · 工程师时间分配



75% 时间在异化劳动

高 V · 低 M

AI 接管

L1+L2+L3

TOMORROW · 工程师时间分配



100% 时间在创造

高 V · 高 M

四个新角色 · 每个团队都需要

01 · ARCHITECT

框架建筑师

设计 L0 引力场
规则即资本

02 · INTENT DESIGNER

意图设计师

写 intent 文件
把业务翻译成声明

03 · HARNESS ENGINEER

外壳工程师

建 Agent 工程外壳
管心跳节奏

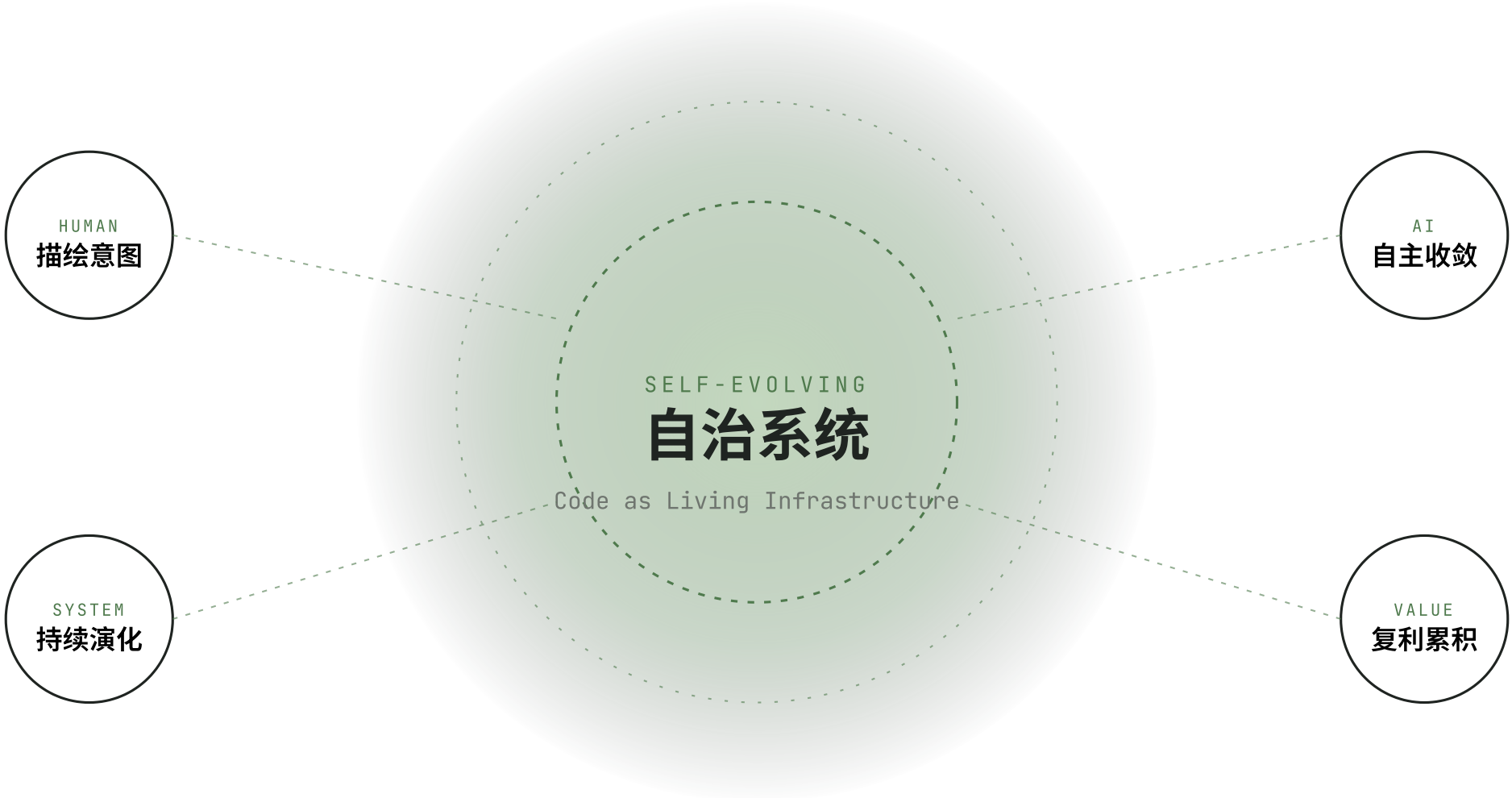
04 · AUDITOR

AI 决策审计员

独立于调优工程师
守 R3-R5 红线

未来形态

2030 • 工程不再是写代码



软件不再"被维护" • 它持续演化。
工程师不再"写代码" • 他们守护一个会自己生长的系统。

价值终章 · 这一切是为了什么

让生产力解放 · 让劳动回归造物

三个主体 · 三种"work"哲学 — AI work everytime · 人 work everywhere & every status

 ENTERPRISE

企业

资产复利 · 反熵增

- **资产健康率**取代代码量 · 成为新北极星
- **交付速度恒定** · 不随代码库年龄衰减
- **故障率↓50%** · onboarding↓60%
- 反哺自己 · 业务专家直接优化系统

COMPOUND · ACCELERATE

 INDIVIDUAL

个人

WORK EVERYWHERE · EVERY STATUS

- **work everywhere** · 工位带到任何地方
- **work every status** · 任何状态都能产出
- **有时间深入业务** · 真正了解用户
- **各行业工作者**都能优化自己的系统 · 反哺企业与国家

LIBERATE · CREATE

 AI

AI

WORK · EVERYTIME

- **7×24** 不休息
- **在沙箱中** 做可逆尝试
- **守门并执行** 而非决策
- **放大人** 而非取代人

AMPLIFY · NEVER REPLACE

AI 不是来替代工程师 · 是来解放工程师

— END · 2026 —